automotive-style Telematics Control Unit (TCU).

What is a TCU?

In a real vehicle, a TCU is responsible for:

- GPS location tracking
- Vehicle diagnostics collection
- Cloud connectivity
- Remote vehicle monitoring
- Emergency services
- OTA updates

Examples:

- Vehicle sends speed/location every second
- Cloud dashboard shows live location
- Mobile app receives vehicle status
- Remote commands sent back to vehicle

Libraries Used

Install:

```
None
sudo apt update

sudo apt install build-essential

sudo apt install cmake

sudo apt install mosquitto

sudo apt install mosquitto-clients
```

```
sudo apt install gpsd gpsd-clients

sudo apt install libgps-dev

sudo apt install bluetooth
```

Hardware Required

Mandatory

- Raspberry Pi 4B
- MicroSD card
- USB WiFi (or onboard WiFi)
- Internet connection

Optional but Recommended

- NEO-6M GPS Module
- HC-05 Bluetooth module

Total cost:

₹700–₹1200

==========

Running C++ Program in PI4b (Linux)

1. Install C++ Compiler

Open Terminal and run:

```
None
sudo apt update
sudo apt install g++
```

Verify:

```
None
g++ --version
```

Create a test program in nano

```
None
nano test.cpp
```

Paste:

```
None
#include <iostream>

int main()
{
   std::cout << "Hello from Raspberry Pi!" << std::endl;
   return 0;
}
```

Save:

```
None
Ctrl + O
Enter
```

```
Ctrl + X
```

Compile:

```
None
g++ test.cpp -o test
```

Run:

```
None
./test
```

Expected output:

```
None
Hello from Raspberry Pi!
```

Very important after any changes made to the .cpp code file updation , deletion of lines , we should

Again

Compile and Run the code , to update the changes

==========
==========

Nano is only a text editor. It creates normal files in Linux, so you can view, edit, move, compile, run, and delete them like any other file.

1. To see Current Location

Check which folder you're in:

```
None
pwd
```

Example output:

```
None
/home/aniruddhan
```

2. View All Files in Current Directory

```
None
ls
```

3. Open a File Again

To edit:

```
None
nano test.cpp
```

4. View File Without Editing

Using `cat`:

```
None
cat test.cpp
```

5. Find the Exact Location of a File

```
None
find ~ -name "vehicle_sim.cpp"
```

Example:

```
None
/home/aniruddhan/vehicle_sim.cpp
```

Find all `.cpp` files:

```
None
find ~ -name "*.cpp"
```

---

6. Delete a Source File

Delete the `.cpp` file:

```
None
rm vehicle_sim.cpp
```

Delete `test.cpp`:

```
None
rm test.cpp
```

---

7. Delete Executable File

Delete compiled program:

```
None
rm vehicle_sim
```

Delete test executable:

```
None
rm test
```

---

8. Delete Both Together

```
None
rm vehicle_sim.cpp vehicle_sim
```

==========
==========

TCU simulation code

Create:

```
None
nano vehicle_sim.cpp
```

Paste:

```
None
#include <iostream>
#include <cstdlib>
#include <ctime>

struct GPSData
{
   double latitude;
   double longitude;
};
```

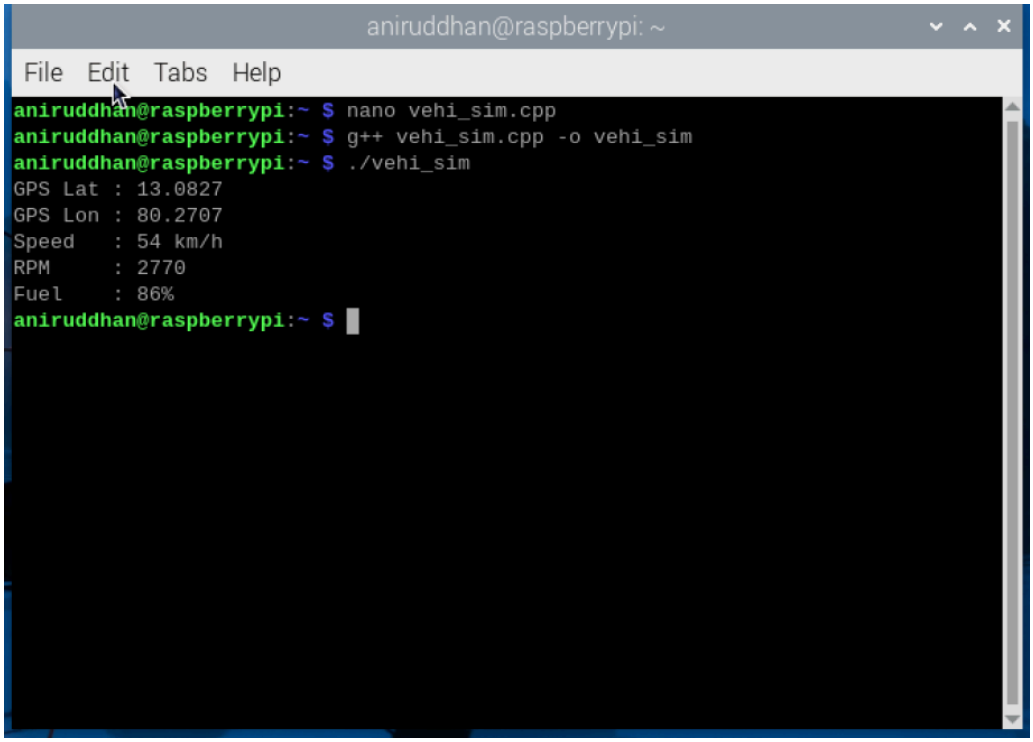
```
int main()
{
   srand(time(NULL));

   GPSData gps;

   gps.latitude = 13.0827;
   gps.longitude = 80.2707;

   int speed = rand() % 120;
   int rpm   = rand() % 6000;
   int fuel  = rand() % 100;

   std::cout << "GPS Lat : " << gps.latitude << std::endl;
   std::cout << "GPS Lon : " << gps.longitude << std::endl;
   std::cout << "Speed   : " << speed << " km/h" << std::endl;
   std::cout << "RPM     : " << rpm << std::endl;
   std::cout << "Fuel    : " << fuel << "%" << std::endl;

   return 0;
}
```

Compile:

```
None
g++ vehicle_sim.cpp -o vehicle_sim
```

Run:

```
None
./vehicle_sim
```



==========

==========

Continuous vehicle simulation code :

None

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <unistd.h>

struct GPSData
{
   double latitude;
   double longitude;
};

int main()
```

```
{
   srand(time(NULL));

   GPSData gps;
   gps.latitude = 13.0827;
   gps.longitude = 80.2707;

   while(true)
   {
       int speed = rand() % 120;
       int rpm   = rand() % 6000;
       int fuel  = rand() % 100;

       gps.latitude += 0.0001;
       gps.longitude += 0.0001;

       std::cout << "\n----- Vehicle Data -----\n";
       std::cout << "Latitude : " << gps.latitude << std::endl;
       std::cout << "Longitude: " << gps.longitude << std::endl;
       std::cout << "Speed    : " << speed << " km/h" << std::endl;
       std::cout << "RPM      : " << rpm << std::endl;
       std::cout << "Fuel     : " << fuel << "%" << std::endl;

       sleep(1);
   }

   return 0;
}
```

Compile:

```
None
g++ vehicle_sim.cpp -o vehicle_sim
```

Run:

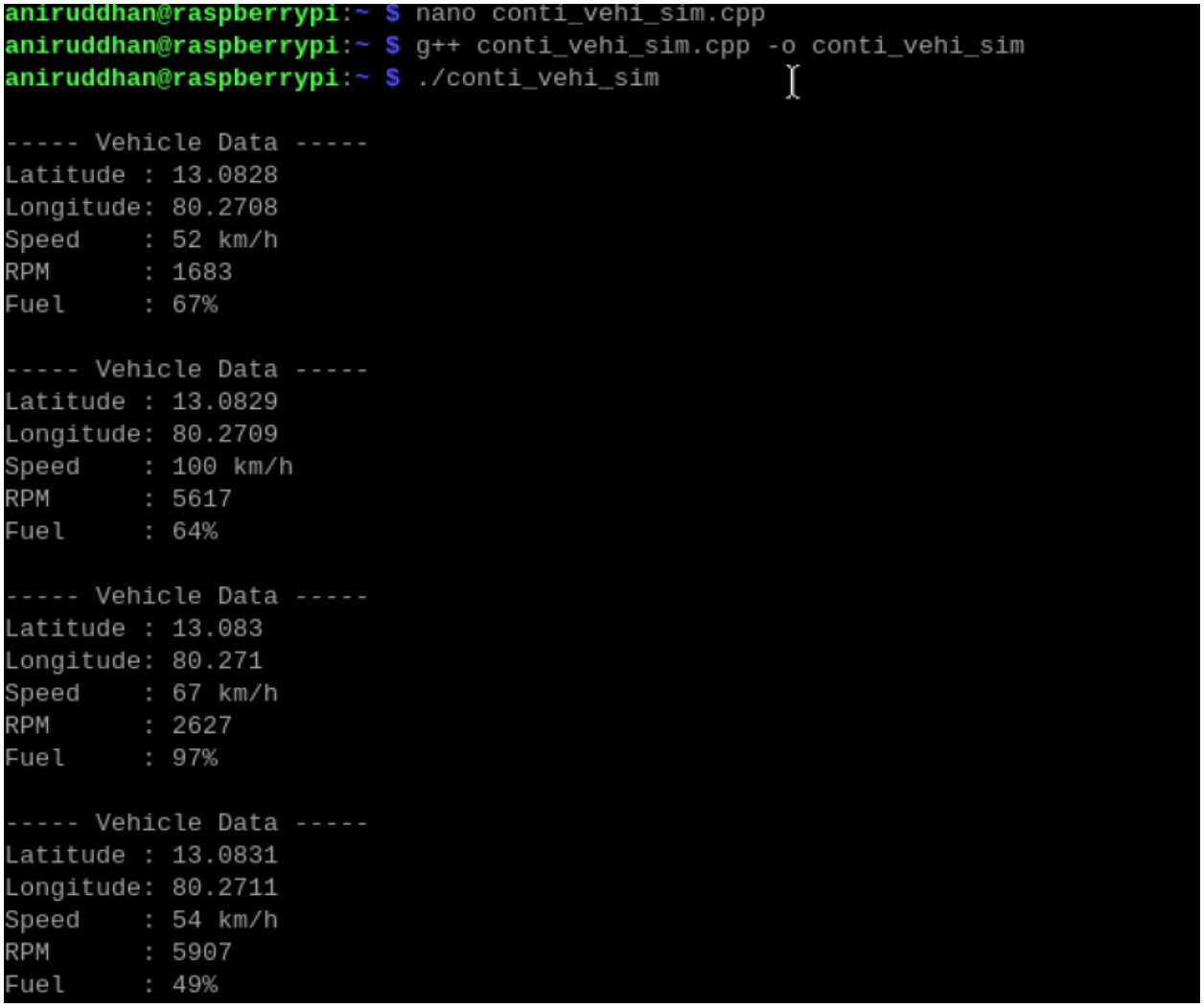
None

```
./vehicle_sim
```

Stop with:

None

```
Ctrl + C
```



==========
==========

Fully Software-Based TCU Simulator

This gives you:

- Embedded Linux
- C++
- MQTT
- TCP/IP
- UDP
- Bluetooth
- GPS
- Systemd
- Logging
- Dashboard

without spending money.

Then if needed:

- Add GPS module later
- Add CAN hardware later
- Add Bluetooth hardware later

Current Hardware Available

I assume you have:

- Raspberry Pi 4B
- MicroSD Card
- WiFi Internet

That alone is enough.

Component Alternatives

GPS Alternative

Instead of:

```
None
NEO-6M GPS Module
```

Use:

GPSD Replay / GPS Simulator

Create a file:

```
None
gps_data.txt



13.0827,80.2707

13.0828,80.2708

13.0829,80.2709
```

Your C++ application reads coordinates periodically.

OR

Use real online GPS feed APIs later.

HC-05 Alternative

You do not need HC-05.

Raspberry Pi already contains:

```
Bluetooth

WiFi
```

Check:

```
bluetoothctl list
```

If adapter appears:

```
Controller XX:XX:XX
```

you already have Bluetooth hardware.

Phase 1

Vehicle Simulator

Generate:

```
Speed
```

```
RPM

Fuel

Engine Temp
```

Example:

```
None
speed = rand()%120;

rpm = rand()%6000;

fuel = rand()%100;
```

This simulates ECU signals.

Phase 2

GPS Simulator

Instead of GPS module:

```
None
struct GPSData

{

   double latitude;
```

```
    double longitude;
};
```

Generate:

```
None
13.0827

13.0828

13.0829
```

every second.

Vehicle appears moving.

Code PHASE 1 AND 2⇒

Conti_vehi_simu.cpp :

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <unistd.h>

struct GPSData
{
```

```
    double latitude;
    double longitude;
};

struct VehicleData
{
    int speed;
    int rpm;
    int fuel;
    int engineTemp;
    GPSData gps;
};

int main()
{
    srand(time(NULL));

    VehicleData vehicle;

    vehicle.gps.latitude = 13.0827;
    vehicle.gps.longitude = 80.2707;
```

```
while(true)
{
    vehicle.speed = rand() % 120;
    vehicle.rpm = rand() % 6000;
    vehicle.fuel = rand() % 100;
    vehicle.engineTemp = 70 + rand() % 40;

    vehicle.gps.latitude += 0.0001;
    vehicle.gps.longitude += 0.0001;

    std::cout << "\n----- Vehicle Data -----\n";

    std::cout << "Latitude     : "
            << vehicle.gps.latitude << std::endl;

    std::cout << "Longitude    : "
            << vehicle.gps.longitude << std::endl;

    std::cout << "Speed        : "
            << vehicle.speed << " km/h" << std::endl;
```

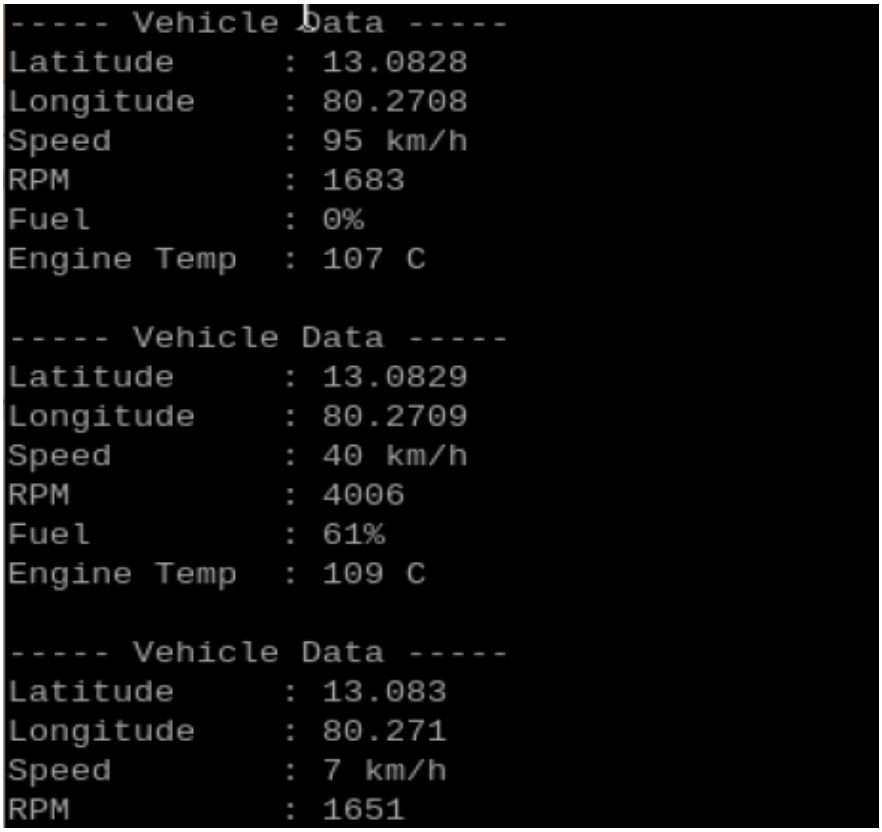
```
        std::cout << "RPM         : "
                  << vehicle.rpm << std::endl;

        std::cout << "Fuel        : "
                  << vehicle.fuel << "%" << std::endl;

        std::cout << "Engine Temp  : "
                  << vehicle.engineTemp << " C" << std::endl;

        sleep(1);
    }

    return 0;
}
```

Linux command used to run the code :

aniruddhan@raspberrypi:~ $ g++ conti_vehi_sim.cpp -o conti_vehi_sim

aniruddhan@raspberrypi:~ $ ./conti_vehi_sim

O/P:



Phase 3.1

MQTT

Install:

```
None
sudo apt update


sudo apt install mosquitto
```

```
sudo apt install mosquitto-clients
```

Topics:

```
None
vehicle/location


vehicle/status


vehicle/diagnostics
```

IMPLEMENTATION

Phase 3 Goal

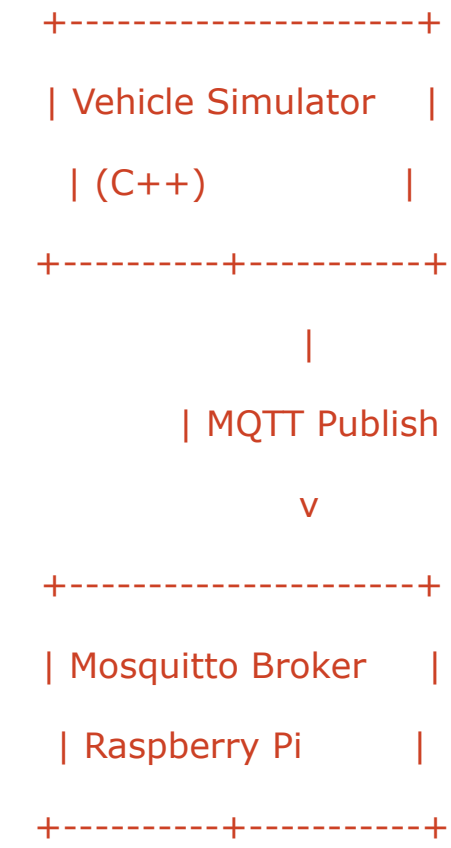
Publish vehicle data to three MQTT topics:

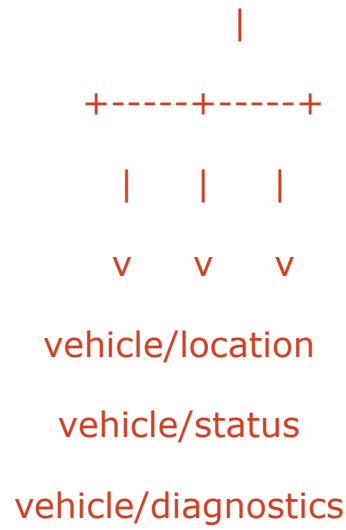
```
None
vehicle/location

vehicle/status

vehicle/diagnostics
```

Data Mapping

Topic	Data
vehicle/location	Latitude, Longitude
vehicle/status	Speed, Fuel
vehicle/diagnostics	RPM, Engine Temperature

Current MQTT architecture:





Verifying the MQTT broker manually before c++MQTT CODING

Terminal 1 (Subscriber)

```
None
mosquitto_sub -h localhost -t vehicle/status
```

Terminal 2 (Publisher)

```
None
mosquitto_pub -h localhost -t vehicle/status -m "Speed=60 Fuel=75"
```

output in Terminal 1:

```
None
Speed=60 Fuel=75
```

Mosquitto is working because

- Publish is working
- Subscribe is working
- We can move directly to integrating MQTT into your vehicle simulator C++ code.

Interesting command :

aniruddhan@raspberrypi:~ $ **dpkg -l | grep mosquitto**

This command lists all the libraries with name mosquitto in it

One More Requirement Before Coding

To publish from C++, we need the Mosquitto development library.

Check if it is installed:

```
None
dpkg -l | grep mosquitto
```

If you do **not** see:

```
None
libmosquitto-dev
```

install it:

```
None
sudo apt update

sudo apt install libmosquitto-dev
```

Then verify:

```
None
ls /usr/include/mosquitto.h
```

Expected:

```
None
/usr/include/mosquitto.h
```

Updated Phase 3 MQTT-enabled TCU Simulator, based directly on your existing code structure.

Publisher Code:

```
None
#include <iostream>

#include <cstdlib>

#include <ctime>

#include <unistd.h>

#include <cstring>

#include <mosquitto.h>
```

```
struct GPSData
{
   double latitude;
   double longitude;
};

struct VehicleData
{
   int speed;
   int rpm;
   int fuel;
   int engineTemp;
   GPSData gps;
};

int main()
{
   srand(time(NULL));
```

```
VehicleData vehicle;

vehicle.gps.latitude = 13.0827;
vehicle.gps.longitude = 80.2707;

// MQTT Initialization
mosquitto_lib_init();

struct mosquitto *mosq =
    mosquitto_new("TCU_Client", true, NULL);

if(!mosq)
{
    std::cerr << "Failed to create MQTT client\n";
    return 1;
}

if(mosquitto_connect(mosq,
```

```
                        "localhost",
                        1883,
                        60) != MOSQ_ERR_SUCCESS)
{
    std::cerr << "Failed to connect to broker\n";
    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    return 1;
}

std::cout << "Connected to MQTT Broker\n";

while(true)
{
    vehicle.speed = rand() % 120;
    vehicle.rpm = rand() % 6000;
    vehicle.fuel = rand() % 100;
    vehicle.engineTemp = 70 + rand() % 40;
```

```
    vehicle.gps.latitude += 0.0001;
    vehicle.gps.longitude += 0.0001;

    // MQTT Messages
    char locationMsg[100];
    char statusMsg[100];
    char diagnosticMsg[100];

    snprintf(locationMsg,
             sizeof(locationMsg),
             "Latitude=%.4f Longitude=%.4f",
             vehicle.gps.latitude,
             vehicle.gps.longitude);

    snprintf(statusMsg,
             sizeof(statusMsg),
             "Speed=%d Fuel=%d",
             vehicle.speed,
             vehicle.fuel);
```

```
snprintf(diagnosticMsg,
         sizeof(diagnosticMsg),
         "RPM=%d EngineTemp=%d",
         vehicle.rpm,
         vehicle.engineTemp);

// Publish Location
mosquitto_publish(
    mosq,
    NULL,
    "vehicle/location",
    strlen(locationMsg),
    locationMsg,
    0,
    false);

// Publish Status
mosquitto_publish(
```

```
        mosq,
        NULL,
        "vehicle/status",
        strlen(statusMsg),
        statusMsg,
        0,
        false);

    // Publish Diagnostics
    mosquitto_publish(
        mosq,
        NULL,
        "vehicle/diagnostics",
        strlen(diagnosticMsg),
        diagnosticMsg,
        0,
        false);

    // Console Output
```

```
        std::cout << "\n----- Vehicle Data -----\n";

        std::cout << "Latitude     : "
                  << vehicle.gps.latitude << std::endl;

        std::cout << "Longitude    : "
                  << vehicle.gps.longitude << std::endl;

        std::cout << "Speed        : "
                  << vehicle.speed << " km/h" <<
std::endl;

        std::cout << "RPM          : "
                  << vehicle.rpm << std::endl;

        std::cout << "Fuel         : "
                  << vehicle.fuel << "%" << std::endl;

        std::cout << "Engine Temp  : "
```

```
                  << vehicle.engineTemp << " C" <<
std::endl;


        sleep(1);
    }


    // Cleanup (not reached in current infinite loop)
    mosquitto_disconnect(mosq);
    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();


    return 0;
}
```

Compile

```
None
g++ tcu_mqtt.cpp -o tcu_mqtt -lmosquitto
```

Run

```
None
./tcu_mqtt
```

Open Subscribers

Terminal 1:

```
None
mosquitto_sub -h localhost -t vehicle/location
```

Terminal 2:

```
None
mosquitto_sub -h localhost -t vehicle/status
```

Terminal 3:

```
None
mosquitto_sub -h localhost -t vehicle/diagnostics
```

You should see live MQTT updates every second while the simulator continues printing the vehicle data to the console.

Note:

Compile with the Mosquitto library:

```
None
g++ tcu_mqtt.cpp -o tcu_mqtt -lmosquitto
```

Notice the extra:

```
None
-lmosquitto
```

This tells the linker:

```
None
Link against libmosquitto.so
```

Phase 3 MQTT Integration is successfully completed. 👍

Terminal 1 (Publisher)

```
None
Connected to MQTT Broker


----- Vehicle Data -----

Latitude     : 13.0828

Longitude    : 80.2708

Speed        : 104 km/h

RPM          : 5128

Fuel         : 73%

Engine Temp  : 103 C
```

This confirms:

- **Vehicle simulator is running ✅**
- **Data generation is working ✅**
- **MQTT client connected to broker ✅**

Terminal 2 (Subscriber)

```
None
Speed=89 Fuel=17

Speed=67 Fuel=97

Speed=57 Fuel=48

...
```

This confirms:

- **Messages are being published to `vehicle/status` ✅**
- **Mosquitto broker is receiving them ✅**
- **Subscribers are receiving them correctly ✅**

==========